Spring 2026 ME/CS/ECE759 Final Project Report

University of Wisconsin-Madison

Software Simulation of Systolic-Array CNN Convolution - Sequential, OpenMP, and CUDA

Mohith Thatikonda

May 5, 2026

Abstract

This final project implements and evaluates a software simulation of a systolic-array CNN convolution dataflow. The project follows the proposed Nim=NPE-style simplification in which the PE grid matches the image size and each PE is responsible for one output pixel. Each PE computes multiple output filters by maintaining one independent accumulator per output channel, representing the multi-MAC-per-PE behavior described in the proposal.

The project delivers a sequential C++ baseline, an OpenMP CPU implementation, a CUDA naive global-memory implementation, a CUDA shared-memory tiled implementation, and a Python reference implementation for validation. All convolution logic is written from scratch without PyTorch, cuDNN, cuBLAS, or other ML libraries. The convolution uses a forward, top-left anchored window: for output position (h,w), the kernel reads input positions (h+kh,w+kw), so boundary padding appears only along the bottom and right edges.

Correctness was validated by running all implementations on identical input.csv and kernel.csv files and comparing every output filter matrix against the Python reference using element-wise numerical tolerance. The documented validation case used H=128, W=128, Cin=16, Cout=8, and K=3; all 32 method/filter comparisons passed with maximum absolute error at or below 1.0e-6, well below the 1e-4 tolerance. Performance was benchmarked on Euler GPU/CPU nodes for 64x64, 128x128, 256x256, and 512x512 images with Cin=16, Cout=8, and K=3. The results show that CUDA kernel execution gives the largest speedup over sequential execution, OpenMP scales well in a separate thread sweep, and the shared-memory CUDA kernel is correct but slightly slower than the naive kernel for the tested 3x3 workload because tile loading and synchronization overhead outweigh the reuse benefit in this configuration.

Link to Final Project git repo: https://github.com/mohith0573/repo759.git

Contents

1. General information

- Name: Mohith Thatikonda
- Email: mthatikonda@wisc.edu
- Home department: Electrical and Computer Engineering
- Status: MS Student
- Teammate: NA; this is an individual project.

- I release the ME759 Final Project code as open source and under a BSD3 license for unfettered use of it by any interested party.

2. Problem statement

The goal of this project was to implement 2D CNN convolution in software while simulating a systolic processing-element dataflow. The project is not a hardware implementation; instead, it models the key dataflow idea in software and evaluates how the computation maps to sequential CPU execution, OpenMP parallel CPU execution, and CUDA GPU execution.

The PE-array model follows the proposal: the image size determines the PE grid, so an H by W image has H by W logical PEs. Each PE computes the output for one spatial pixel. Within each PE, multiple logical MAC accumulators are used, one per output filter. Every accumulator walks through all input channels and all K by K kernel taps, accumulating a separate output value for that filter.

The project intentionally uses a forward, top-left anchored convolution rule instead of a standard centered convolution. For output PE (h,w), the kernel accesses input pixels (h+kh,w+kw). Thus, padding is required only on the bottom rows and right-most columns. This directionality matches the intended data reuse pattern: an input pixel is reused by PEs located above and to the left within the kernel window.

The motivation is that convolution dominates CNN inference cost and naturally exposes both parallelism and data reuse. Neighboring output windows overlap, so this computation is a good vehicle for studying ME759 topics such as OpenMP loop parallelism, CUDA thread/block hierarchy, memory coalescing, shared memory tiling, strong scaling, and roofline-style analysis. The project is inspired by Endrawati et al. (2025), but the work here is a software simulation and performance study rather than a hardware replication.

3. Solution description

Convolution and data layout. All implementations use the same flattened data layout. The input is stored as input[ci][h][w], the kernel as kernel[co][ci][kh][kw], and the output as output[co][h][w]. The flattened index expressions are input index = (ci*H+h)*W+w, kernel index = ((co*Cin+ci)*K+kh)*K+kw, and output index = (co*H+h)*W+w. This layout keeps each input channel contiguous in row-major order and stores each output filter as one output matrix.

Common computation. Every method computes the same equation: output[co][h][w] = sum over ci, kh, and kw of input[ci][h+kh][w+kw] multiplied by kernel[co][ci][kh][kw]. If h+kh or w+kw is outside the image, that term contributes zero. This common computation is the basis for comparing all implementations against the Python reference.

Sequential implementation. The sequential C++ implementation provides the baseline. It loops over output pixels and uses a local accumulator vector for all Cout filters at that PE. This improves

locality compared with recomputing each filter separately, because an input value loaded for a PE can be reused across multiple output-filter accumulators before moving on.

OpenMP implementation. The OpenMP implementation parallelizes across output pixel positions using CPU threads. Each thread has private accumulator storage, avoiding races while still using the same multi-filter accumulation strategy as the sequential code. The OpenMP sweep evaluates 1, 2, 4, 8, 16, and 20 threads.

CUDA naive implementation. The CUDA naive version maps one CUDA thread to one output PE. Threads are arranged as 16 by 16 blocks, giving 256 threads per block. Each thread computes all output filters for one pixel using register accumulators. It reads required input windows directly from global memory, so it is simple and exposes massive parallelism but does not explicitly cache overlapping input windows in shared memory.

CUDA shared-memory implementation. The CUDA shared version uses the same one-thread-per-PE mapping, but each block cooperatively loads a shared-memory tile for each input channel. Because the convolution reads to the right and bottom, the shared tile has size (TILE+K-1) by (TILE+K-1). For TILE=16 and K=3, each block loads an 18 by 18 tile. Threads then reuse shared-memory pixels while accumulating all output filters. This mirrors the proposal goal of reducing redundant external memory access through on-chip reuse.

Python reference and shared inputs. The Python reference uses the same equation and indexing, and it is used as the ground truth. The inputs/ folder generates size-specific input and kernel files. Each implementation receives a copy named input.csv and kernel.csv so that all methods operate on identical data. This avoids the common validation error of comparing methods that used different random inputs.

4. Overview of results. Demonstration of your project

Correctness. The correctness comparison was performed in results_comparison/ for H=128, W=128, Cin=16, Cout=8, and K=3. The script first confirmed that input.csv and kernel.csv were identical across implementation folders. It then compared all eight output filter matrices from sequential, OpenMP, CUDA naive, and CUDA shared against the Python reference. All 32 comparisons passed with tolerance 1e-4. The largest observed max_abs_diff was 1.00e-06. The largest per-method max_abs_diff values were: sequential 6.60e-07, OpenMP 1.00e-06, CUDA naive 7.00e-07, and CUDA shared 7.00e-07.

Benchmark configuration. The main performance benchmarks used H=W = 64, 128, 256, and 512 with Cin=16, Cout=8, and K=3. The benchmark scripts set WRITE_MATRICES=0 so that timing is not dominated by output CSV generation. CPU and Python results report time_ms. CUDA performance plots use kernel_time_ms, while total_time_ms is kept separately because it includes allocation, copies, and other host-side overhead.

Size	Sequential (ms)	OpenMP 20T (ms)	Python ref (ms)
64x64	2.01044	18.9982	444.212
128x128	8.17184	11.4057	1804.29
256x256	33.0402	17.5977	7739.03
512x512	132.626	32.7956	30100.2

Table 1. CPU/Python timing summary. Sequential, OpenMP, and Python columns report time_ms.

Size	CUDA naive kernel (ms)	CUDA shared kernel (ms)
64x64	0.034918	0.054938
128x128	0.113459	0.114374
256x256	0.404634	0.423322
512x512	1.48832	1.53948

Table 2. CUDA kernel timing summary. Both CUDA columns report kernel_time_ms measured with CUDA events.

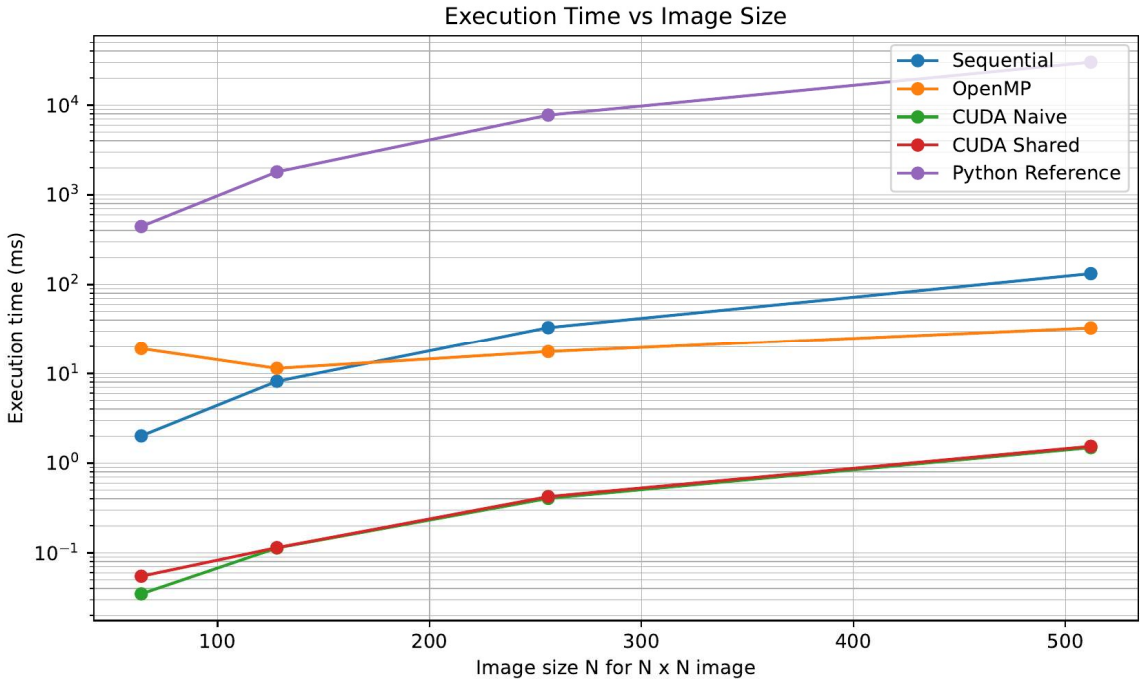

Figure 1. Execution time as image size increases for the implemented methods.

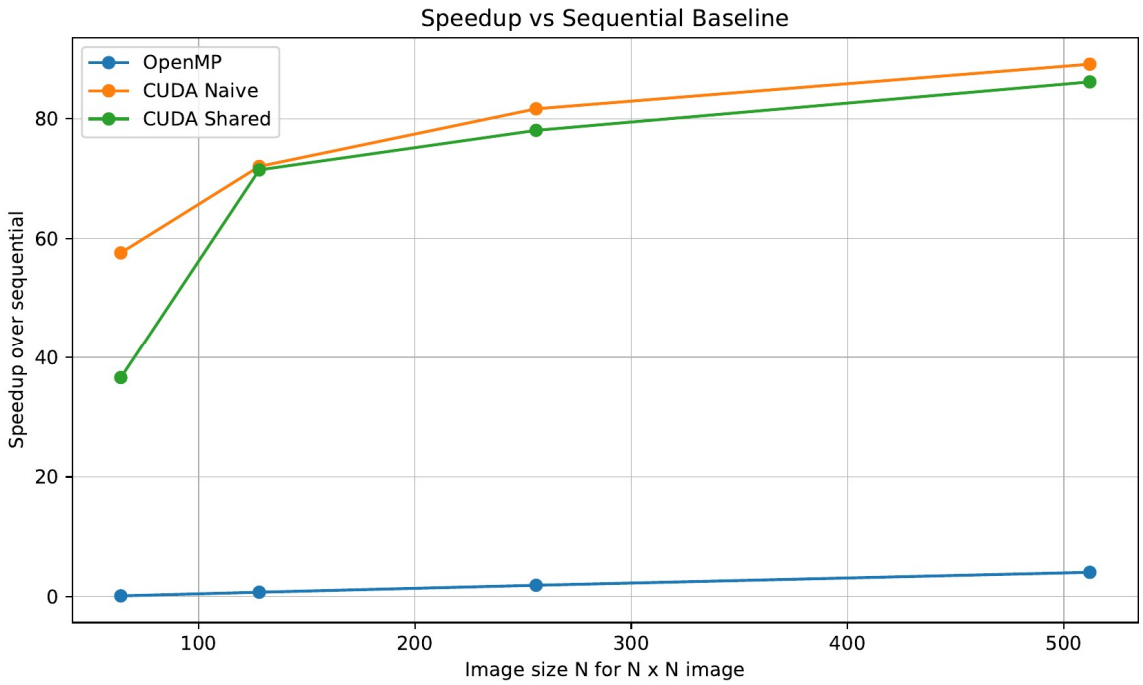

Figure 2. Speedup over the sequential baseline. CUDA provides the largest speedups; OpenMP improves as image size increases.

The benchmark results satisfy the main project objective of comparing sequential, OpenMP, and CUDA approaches. For the 512x512 case, the sequential baseline required 132.63 ms, OpenMP with 20 threads required 32.80 ms, CUDA naive required 1.488 ms of kernel time, and CUDA shared required 1.539 ms of kernel time. This corresponds to approximate speedups of 4.04x for OpenMP, 89.11x for CUDA naive, and 86.15x for CUDA shared over the sequential baseline.

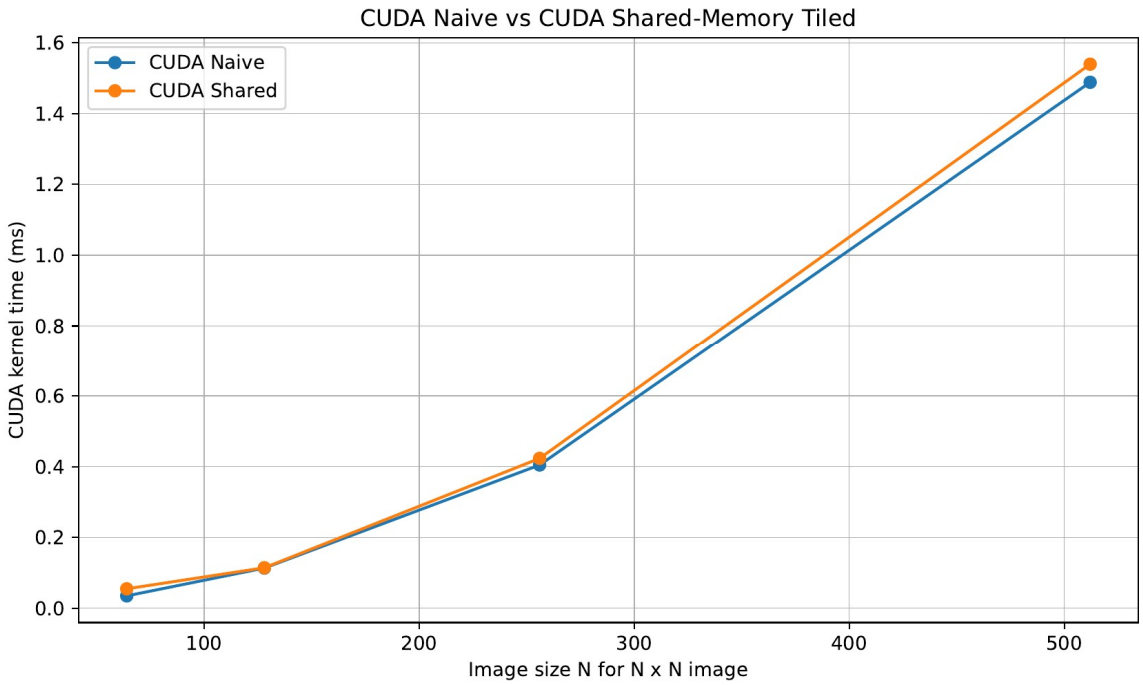

Figure 3. CUDA naive global-memory kernel compared with CUDA shared-memory tiled kernel.

The shared-memory implementation produced correct results but was slightly slower than the naive CUDA implementation for the tested K=3 workload. This result is reasonable: the shared-memory kernel performs extra cooperative tile loading, halo handling, and synchronization. For the tested sizes and 3x3 kernel, the reuse benefit was not large enough to overcome this overhead. This is an important HPC observation because an optimization can be correct and theoretically motivated but still not improve performance for every operating point.

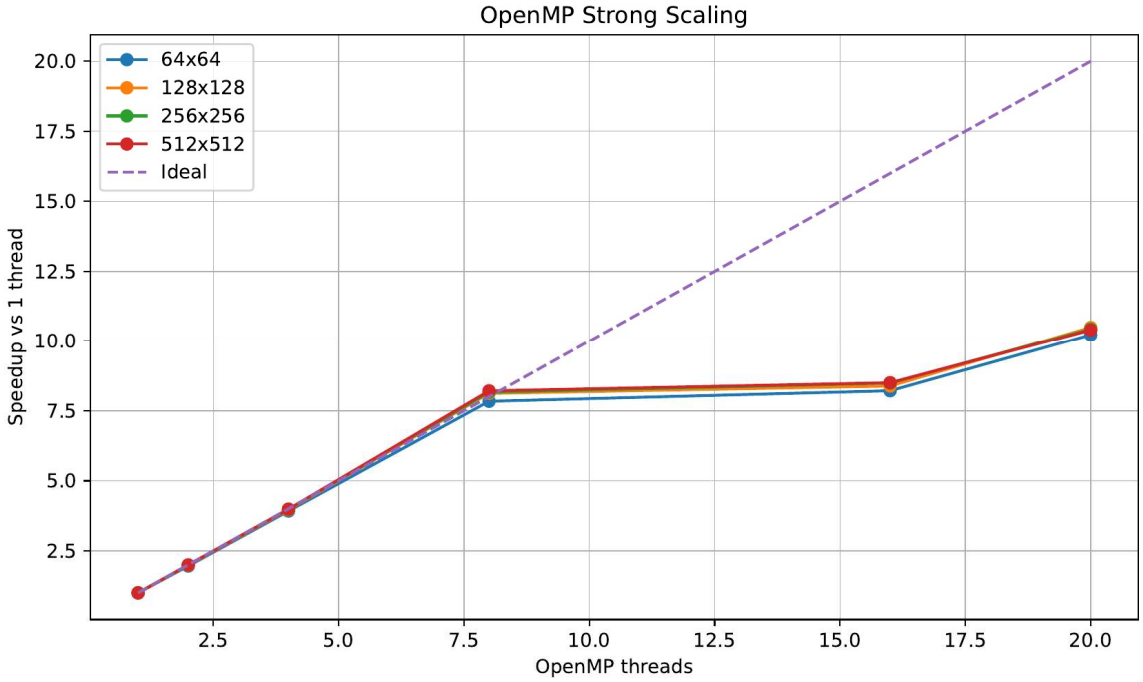

Figure 4. OpenMP strong scaling for 1, 2, 4, 8, 16, and 20 threads.

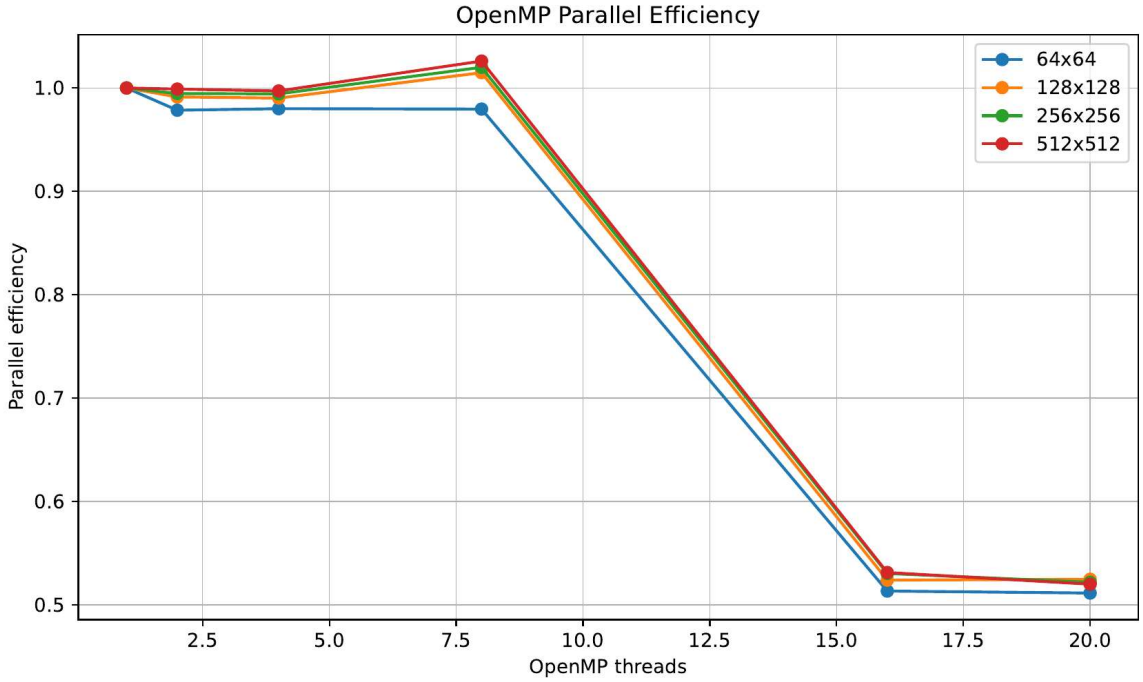

Figure 5. OpenMP parallel efficiency as thread count increases.

The OpenMP sweep shows strong scaling for the fixed-size workloads. For the 512x512 case, runtime decreased from 162.83 ms at one thread to 15.66 ms at 20 threads, giving about 10.4x speedup. Efficiency is close to ideal through 8 threads in the measured results and then drops at 16

and 20 threads, which is consistent with thread scheduling overhead, finite work per thread, and memory bandwidth effects.

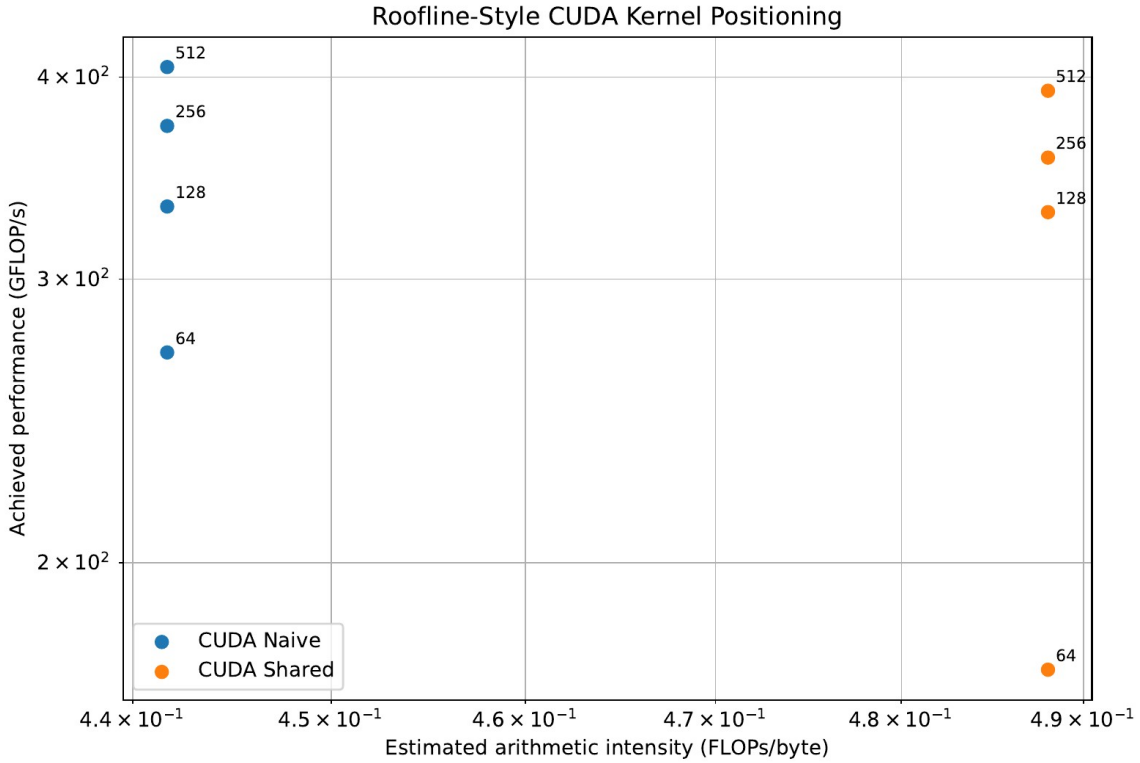

Figure 6. Roofline-style CUDA positioning using analytical arithmetic intensity and achieved GFLOP/s.

The roofline-style plot estimates FLOPs as 2*H*W*Cin*Cout*K*K. The factor of two counts one multiply and one add for each MAC operation. The arithmetic-intensity values are analytical estimates, not direct Nsight measurements, but they help show how the CUDA kernels occupy the performance space. The shared-memory kernel has a slightly higher estimated arithmetic intensity because it reduces redundant input tile reads, even though its measured kernel time is slightly higher for this workload.

5. Deliverables. Building & running your project

The final project repository is organized as follows:

- inputs/: generates and stores size-specific input and kernel CSV files, such as input_64.csv, kernel_64.csv, input_128.csv, and kernel_128.csv.
- sequential_execution/: sequential C++ implementation, Makefile, SLURM script, and sample output matrices/results.
- openmp/: OpenMP implementation, Makefile, SLURM script, and generated results.
- cuda_naive/: CUDA global-memory implementation and GPU SLURM script.
- cuda_shared/: CUDA shared-memory tiled implementation and GPU SLURM script.
- python_ref/: Python reference implementation used for validation.

- results_comparison/: element-wise comparison scripts and validation CSVs.
- exe_results/: benchmark timing CSV files for 64, 128, 256, and 512 image sizes.
- plots/: plotting scripts, OpenMP sweep utilities, generated PDF plots, and benchmark table.

Generating inputs. From inputs/, run the generation SLURM script or Python generator to create size-specific input and kernel files. For each implementation run, the appropriate input_N.csv and kernel_N.csv files are copied into the target implementation directory as input.csv and kernel.csv. This naming convention ensures that all implementations use the same data for a given benchmark.

Running implementations. Each implementation directory contains a SLURM script that compiles and runs that method. The CPU implementations use g++ with -O3 and, for OpenMP, -fopenmp. The CUDA implementations use nvcc with nvidia/cuda/13.0.0 on the instruction GPU partition. The scripts were written in the same style as ME759 assignments and run on Euler.

Typical commands are:

cd sequential_execution && sbatch sequential_job.sh

cd openmp && sbatch openmp_job.sh

cd cuda_naive && sbatch cuda_naive_job.sh

cd cuda_shared && sbatch cuda_shared_job.sh

cd python_ref && sbatch python_job.sh

cd results_comparison && sbatch comparison_job.sh

cd plots && sbatch plots_job.sh

cd plots && sbatch openmp_sweep_job.sh && sbatch plot_openmp_sweep_job.sh

Reproducing reported plots. The exe_results/ folder stores the timing CSVs used by the plotting scripts. Running plots/plots_job.sh regenerates the benchmark table, execution-time plot, speedup plot, CUDA comparison plot, total-time plot, and roofline-style plot. Running the OpenMP sweep scripts in plots/ regenerates the strong-scaling and efficiency figures.

Deliverable status. The delivered code accomplishes the proposal goals: sequential baseline, OpenMP implementation with thread sweep, CUDA naive kernel, CUDA shared-memory tiled kernel, Python reference validation, benchmark CSVs, plots, roofline-style analysis, and README/build/run documentation. The kernel-size and channel-count dimensions are represented by the implemented code and current Cin=16 benchmark; future work could expand the benchmark matrix further to include K=5 and K=7.

6. Conclusions and Future Work

This project successfully implemented a software simulation of a systolic-array-inspired CNN convolution dataflow and compared sequential, OpenMP, CUDA naive, CUDA shared-memory, and Python reference versions. The most important result is that all implementations produce matching output feature maps when run on identical inputs and kernels. The element-wise comparison against Python reference passed for every filter and every implementation in the validation case.

The performance results highlight several ME759 lessons. First, the amount of useful parallel work matters: OpenMP speedup improves as image size grows, and the dedicated sweep shows strong

scaling across thread counts. Second, CUDA exposes much higher parallelism for this output-pixel mapping and achieves the largest speedups over sequential. Third, memory hierarchy optimizations must be evaluated empirically. The shared-memory CUDA kernel models the intended tile reuse, but for the tested 3x3 workload its overhead slightly exceeds its benefit compared with the naive kernel. This is a useful result because it shows that optimization effectiveness depends on problem size, kernel size, reuse level, and synchronization overhead.

Future work would include expanding the benchmark space to K=5 and K=7, sweeping more input-channel and output-channel counts, adding more precise Nsight Compute profiling, and exploring optimized CUDA variants that distribute output channels across threads or use constant memory for filter weights. The current project already leverages major ME759 topics: OpenMP parallel loops and strong scaling, CUDA grid/block/thread hierarchy, shared memory tiling, memory coalescing considerations, CUDA event timing, and roofline-style performance reasoning.

References

[1] D. N. Endrawati, S. Dharma, I. Syafalni, N. Sutisna, T. Adiono, and H. Kunieda, “Zigzag Dataflow Architecture for Convolutional Neural Network,” in 2025 IEEE 38th International System-on-Chip Conference (SOCC), 2025, doi: 10.1109/SOCC66126.2025.11235436.

[2] NVIDIA, CUDA C++ Programming Guide. Used as reference background for CUDA thread hierarchy, global memory, and shared memory concepts.

[3] OpenMP Architecture Review Board, OpenMP API Specification. Used as reference background for OpenMP parallel loop execution and thread-level CPU parallelism.

[4] ME/CS/ECE 759 course materials, Spring 2026. Used for concepts including CUDA execution, OpenMP scaling, timing, profiling, and roofline-style analysis.